MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

Semlalia
Faculté des Sciences Semlalia
Département de L'informatique

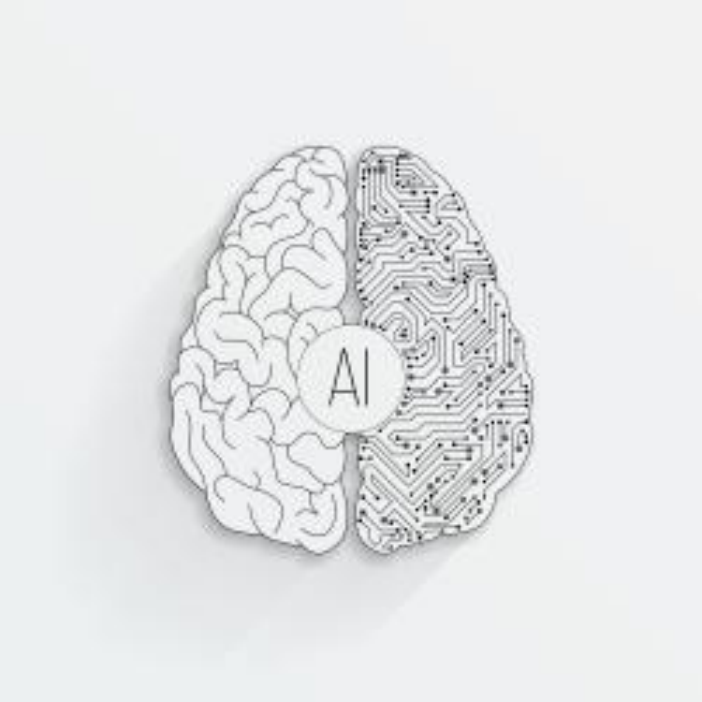
python for AI

Python pour l'Intelligence Artificielle
Chapitre 1 : Environnement de développement

Licence IA SD - S5

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Définition de l'IA



qu'est ce que l'intelligence artificielle ?

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Définition de l'IA

Marvin Minsky (1968):

L'IA a pour but la construction de programmes informatiques qui s'adonnent à des tâches qui sont, pour l'instant, accomplies de façon plus satisfaisantes par des êtres humains car elles demandent des processus mentaux de haut niveau tels que: l'apprentissage perceptuel, l'organisation de la mémoire et le raisonnement critique.

E. Feigenbaum

"AI is the part of computer science concerned with designing intelligent computer systems"

(IA est la partie de l'Informatique qui s'intéressent à la conception des systèmes intelligents)

Définition de l'IA

Autres …. récoltées sur le Net

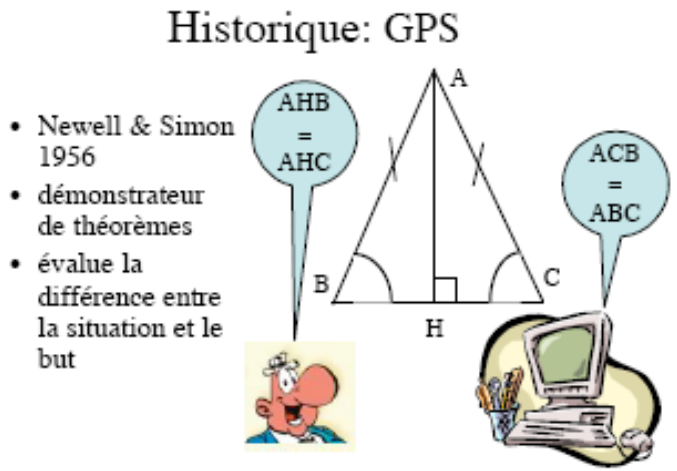
- IA est la simulation des processus intelligents chez **l'humain**
- IA est la **reproduction des méthodes** ou des résultats du raisonnement ou de l'intuition **humains**
- IA est l'étude des facultés mentales par des ***méthodes* calculatoires** et informatiques
- Utiliser des **modèles calculatoires** pour simuler un **comportement intelligent**
- Des machines imitant les **humains**
- ……..

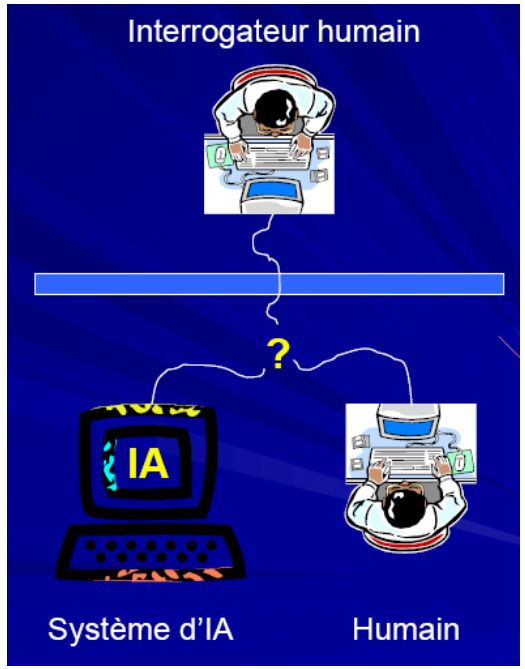
Définition de l'IA

- Définition de l'IA

Penser comme des humains

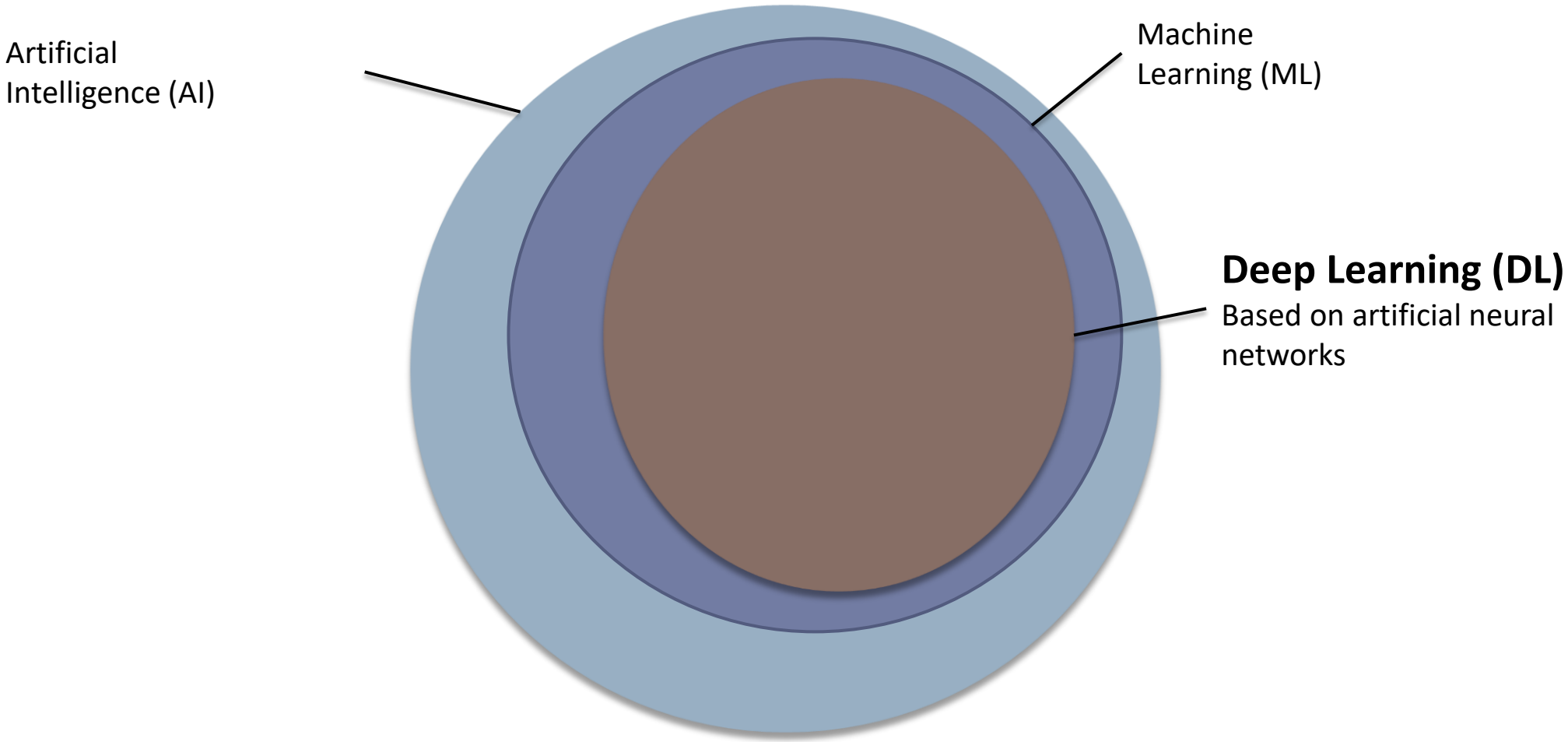
Agir comme des humains





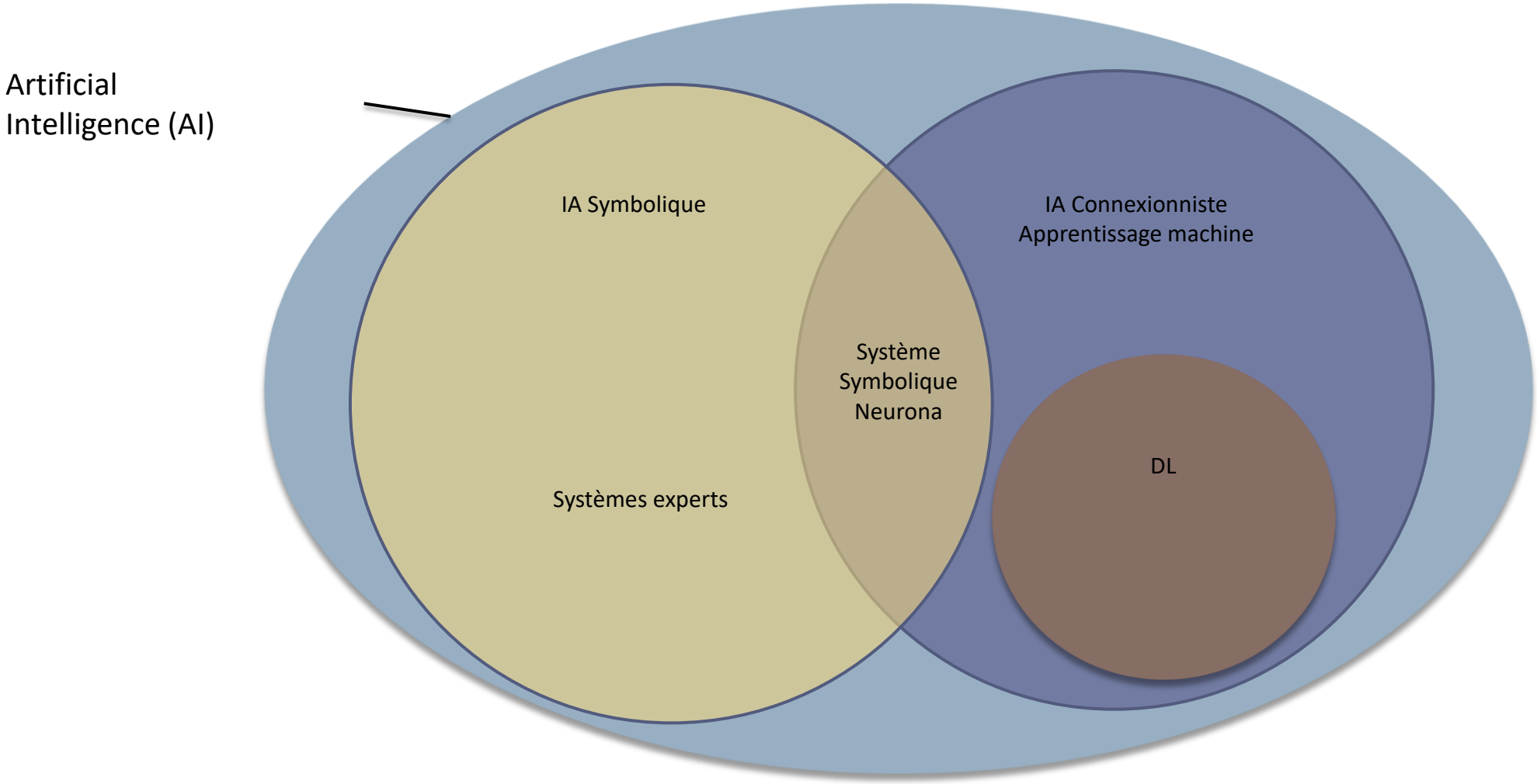
Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

IA



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

IA



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

IA

[**intelligence**] « Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et rationnelle »:

IA symbolique (ou IA logique) :

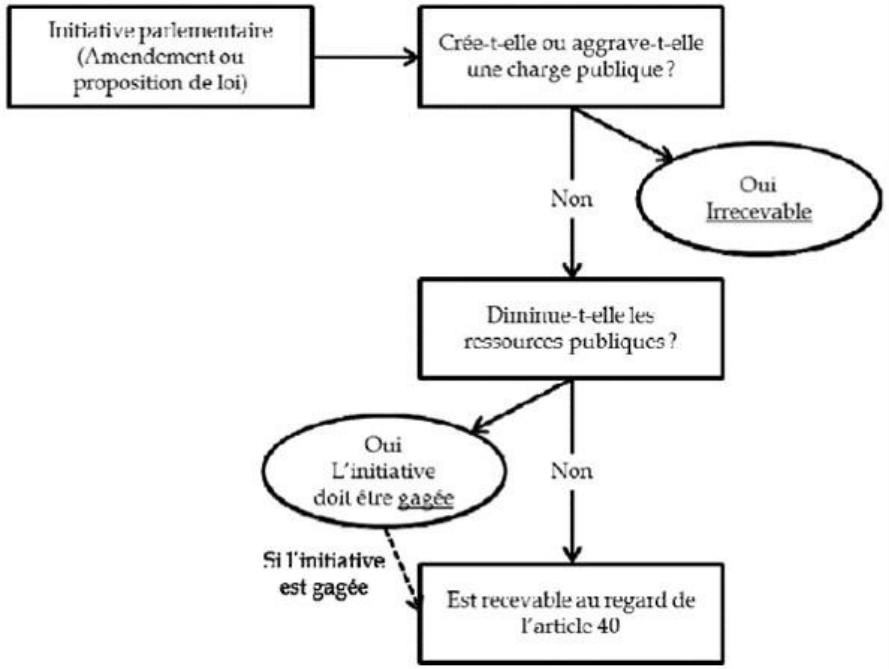
- L'IA symbolique repose sur des **règles** explicites, des symboles et de la logique formelle pour représenter les connaissances et résoudre des problèmes. Elle fonctionne par raisonnement logique (ex : si… alors…), et est utilisée dans les systèmes experts, la planification ou encore les moteurs d'inférence.

IA connexionniste :

- L'IA connexionniste s'inspire du fonctionnement du cerveau humain et repose sur des **réseaux de neurones** artificiels. Elle apprend à partir des données en ajustant automatiquement ses connexions internes, comme dans le Machine Learning et le Deep Learning, sans règles explicites préprogrammées.

IA connexionniste vs IA symbolique

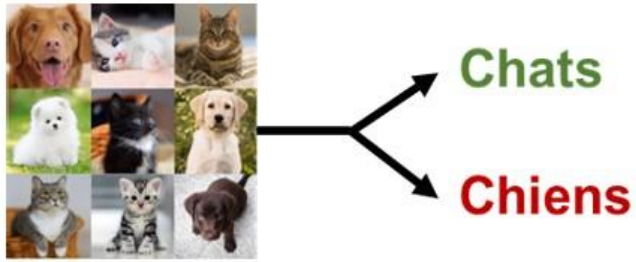
Algorithme classique (IA symbolique)



Recevabilité d'un amendement (art. 40 de la Constitution)

Apprentissage automatique = IA

Phase **1** : entraînement



Phase **2** : utilisation

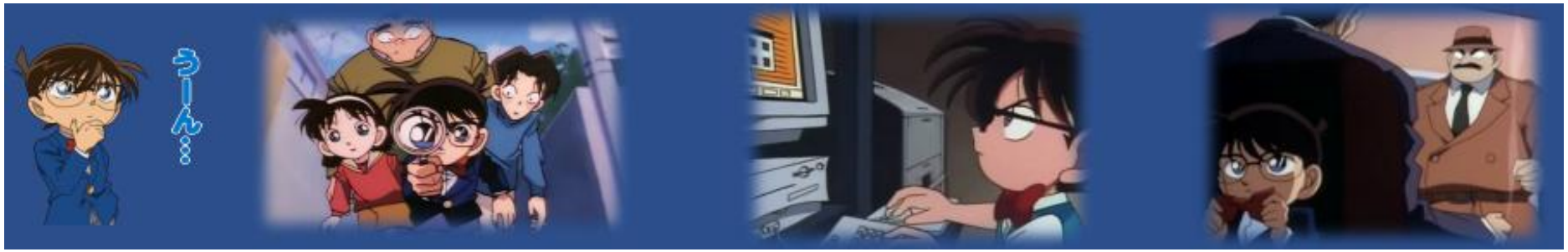
Reconnaissance visuelle (chat) par apprentissage automatique supervisé

Qu'est-ce que la science des données ?

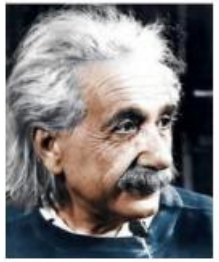
- La science des données est une discipline qui consiste à utiliser les **données** pour résoudre des problèmes complexes et prendre des décisions informées
- La science des données est un art qui transforme les données brutes en informations significatives (utiles).
- Cela implique la collecte, l'analyse et l'interprétation de données pour découvrir des tendances, faire des prédictions et pour la prise de décision.
- C'est le mélange de compétences en programmation, en statistiques et en expertise métier pour résoudre des problèmes complexes à partir de données.

Comment fonctionne la science de données ?

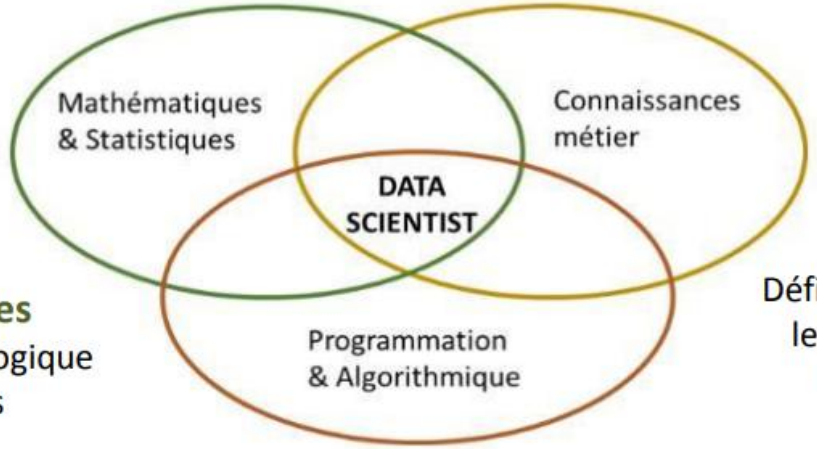
- La science des données est une discipline qui fonctionne en suivant un processus structuré de collecte, traitement, analyse et interprétation des données.
- Elle repose sur un ensemble d'étapes clés pour extraire des informations exploitables à partir des données.
- C'est un peu comme un détective qui cherche à résoudre un mystère en utilisant des indices.



Data Scientist ?



Maitrise des méthodes
Définit l'approche méthodologique
et Réalise les analyses



Sens Business
Définit le besoin, Exploite
les résultats et met en
œuvre les actions

« Une personne meilleure en statistique que n'importe quel développeur et meilleure en développement que n'importe quel statisticien »

— Josh Wills

Maitrise des Outils (IT)
Nettoie, Optimise, offre
l'infrastructure IT nécessaire,
Implémente la solution

IA + DS

Générer de la valeur avec la data et l'IA

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Développement de l'IA

- La programmation traditionnelle et le développement de l'IA adoptent des approches fondamentalement différentes pour **résolution de problèmes**.
- Le codage traditionnel repose sur **règles et instructions clairement définies**, où un développeur écrit **algorithmes précis** pour traiter les entrées et générer des sorties.
- En revanche, l'IA est **pilotée par les données**, ce qui signifie que **apprend des modèles** et prend des décisions sur la base de grands ensembles de données sans nécessiter de programmation explicite pour tous les scénarios possibles.

Développement de l'IA

Caractéristique	Programmation Traditionnelle	Développement de l'IA
Approche	Basée sur des règles	Pilotée par les données
Capacité d'apprentissage	Pas d'apprentissage ; suit des instructions explicites	Apprend des données et s'adapte
Gestion des données non structurées	Limitée	Excellente (texte, images, vidéos)
Flexibilité	Nécessite des mises à jour manuelles pour de nouveaux scénarios	S'adapte automatiquement avec de nouvelles données
Demande de calcul	Modérée	Élevée (nécessite souvent des GPU)
Exemples	Logiciels bancaires, calculatrices, gestion des stocks	Reconnaissance d'images, traitement de la parole, voitures autonomes

Pourquoi Python pour IA ?

Importance de Python en IA

La popularité de Python dans le domaine de l'IA repose sur plusieurs facteurs :

- **Simplicité** : La syntaxe claire et intuitive de Python le rend facile à apprendre et à utiliser → ce qui est crucial dans le domaine dynamique et rapide de l'IA.
- **Polyvalence** : La polyvalence de Python permet aux développeurs de s'attaquer à un large éventail de tâches en IA, allant du prétraitement des données et de l'entraînement des modèles jusqu'au déploiement et à la production → Python propose des bibliothèques et des outils adaptés.
- **Écosystème riche** : Python dispose d'un vaste écosystème de bibliothèques, frameworks et outils qui simplifient le processus de développement. De TensorFlow et Keras pour l'apprentissage profond à Scikit-learn et PyTorch pour l'apprentissage automatique, Python offre un accès aux technologies et algorithmes de pointe, permettant aux développeurs de construire des systèmes d'IA puissants et sophistiqués.
- **Soutien communautaire** : La communauté dynamique et active de développeurs, chercheurs et passionnés autour de Python contribue à son amélioration continue et à son évolution. Grâce à une multitude de ressources, forums, tutoriels et projets open sources disponibles en ligne, les développeurs Python bénéficient d'un soutien précieux et de nombreuses opportunités de collaboration, favorisant l'innovation et le partage des connaissances dans le domaine de l'IA.

Pourquoi Python pour IA ?

Caractéristiques principales de Python pour l'IA :

Python offre plusieurs caractéristiques qui le rendent particulièrement adapté au développement en IA :

- **Typage dynamique** : Le typage dynamique de Python permet un code flexible et concis, permettant aux développeurs de se concentrer sur la résolution de problèmes liés à l'IA sans être ralentis par des détails de bas niveau.
- **Structures de données de haut niveau** : Python fournit des structures de données avancées telles que les listes, dictionnaires, ensembles et tuples, qui simplifient la manipulation et l'analyse des données.
- **Langage interprété** : Python est un langage interprété, ce qui signifie que l'exécution du code se fait ligne par ligne, le rendant idéal pour le développement interactif et le prototypage rapide. Cette nature interactive permet aux développeurs d'expérimenter avec des algorithmes, de visualiser les résultats et d'itérer rapidement, accélérant ainsi le cycle de développement de l'IA.
- **Bibliothèque standard étendue** : La vaste bibliothèque standard de Python offre un large éventail de modules et de fonctions pour réaliser des tâches courantes telles que la gestion de fichiers, le réseau et le traitement de données. Cette richesse de fonctionnalités intégrées accélère le développement en IA en fournissant des solutions prêtes à l'emploi aux défis fréquents.

Rappel

- Avant de se concentrer sur les outils, un petit rappel de ce dont on a besoin pour coder en Python :

  - Une ou plusieurs **versions de Python**
  - Des **packages** et leurs **dépendances**
  - Une **IDE** bien adaptée
  - Parfois des codes / packages dans d'autres langages (par exemple **Cuda pour le deep learning sur GPU**)
  - Des **environnements virtuels** pour gérer les packages associés à chaque projet

- Pour répondre à toutes ces contraintes, il faut vous équiper d'un certain nombre d'outils et chacun ont leurs propres spécificités.

Définitions

- **Un package** (Paquet) : on l'appelle aussi bibliothèque, librairie... Il s'agit d'un ensemble de classes, de fonctions ou de modules qui travaillent ensemble pour fournir des fonctionnalités spécifiques qui ne sont pas disponibles dans le langage Python.

- **Une dépendance** : ceci fait référence à un autre package ou ensemble de packages nécessaires pour qu'un package spécifique fonctionne correctement.

- **Un environnement virtuel** : c'est un outil utilisé pour créer des espaces isolés pour les projets Python, où vous pouvez gérer séparément les dépendances et les versions des packages pour chaque projet.

Gestion des paquets

- Module Python : On appelle “module” tout fichier constitué de code Python (c’est-à-dire tout fichier avec l’extension .py contenant un ensemble de fonctions, de classes et de variables prédéfinies) importé dans un autre fichier ou script.
- En classant selon leur **éditeur**, on peut distinguer trois grandes catégories de modules :
  - les modules standards (ils sont intégrés automatiquement par Python, mais ne font pas réellement partie du langage),
  - les modules qui sont développés par le programmeur,
  - les modules développés par un développeur externe.
- Un **package** est une collection, un **ensemble de modules Python**.
- Un package est simplement un dossier contenant plusieurs fichiers Python (.py) et un fichier additionnel nommé __init__.py

Gestion des paquets

Module Python

geometry.py

Classes

```
class carre:
    ....

class triangle:
    ....

class cercle:
    ....
```

Variables

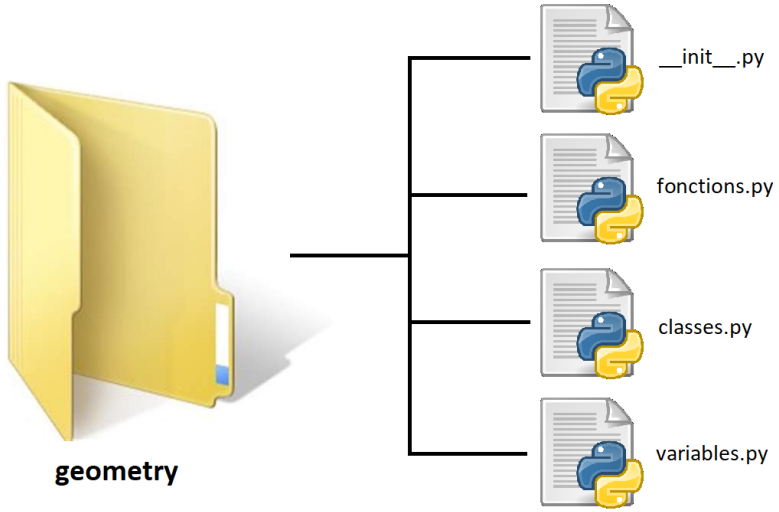
```
pi = 3.14159265359
phi = 1.61803398875
```

Fonctions

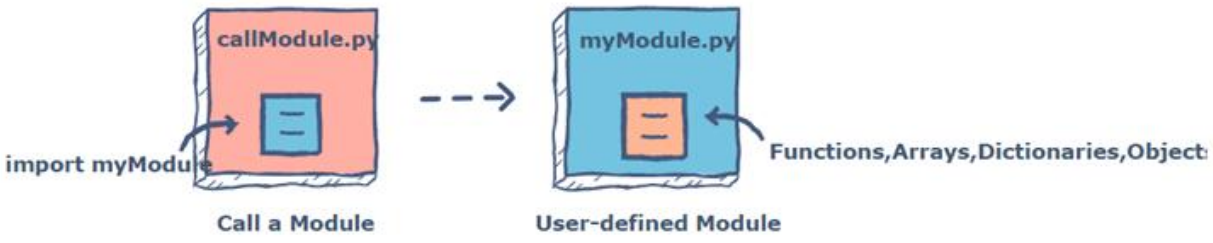
```
def aire(geometry_object):
    ....

def angles(triangle):
    ....
```

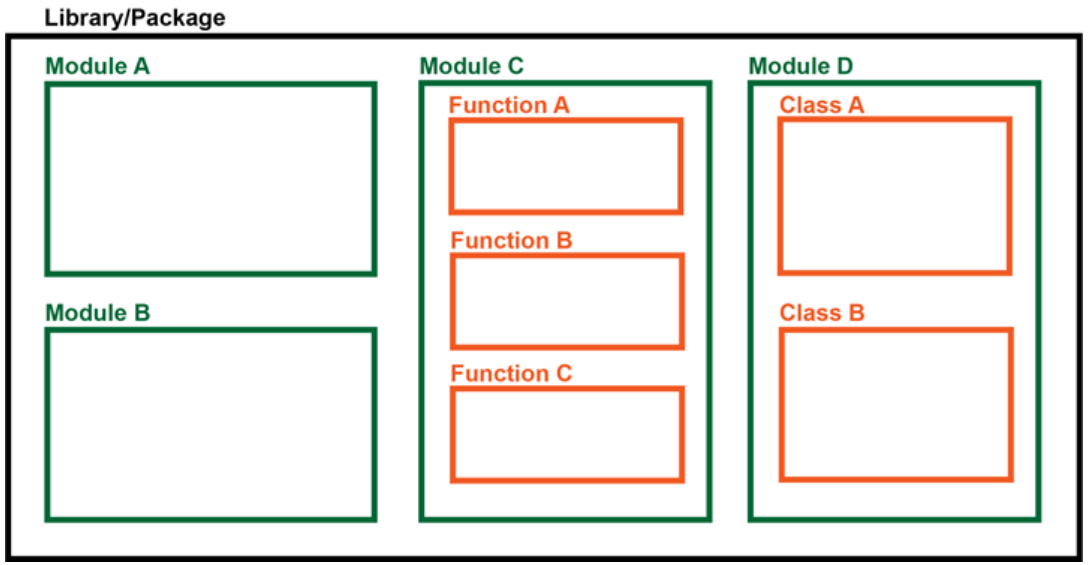
package

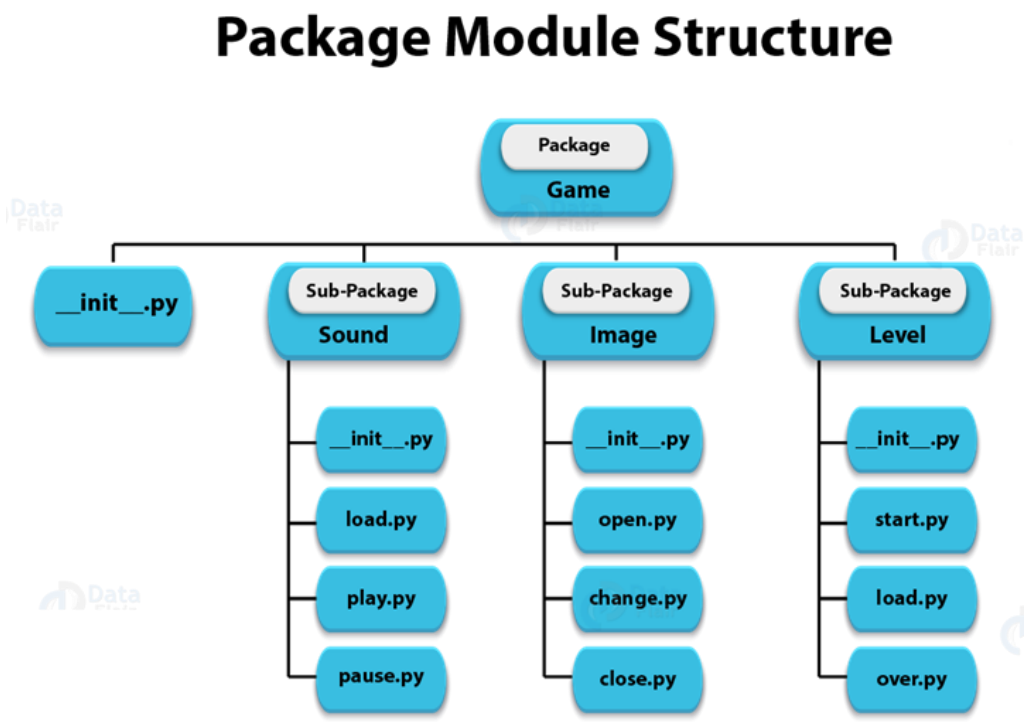


Gestion des paquets



Module





Package

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Gestion des paquets

Pour importer un package, il faut tout d'abord qu'il soit installé sur la machine. Ensuite, il faut utiliser les mots-clés suivants:

`import package as alias` ou `from package import subpackage as alias`

Exemple :

Syntaxe 1 : Importer tout le package avec un alias

```
import numpy as np
```

Cela vous permet d'utiliser toutes les fonctionnalités de numpy en écrivant simplement np.

Par exemple :

```
import numpy as np
resultat = np.sqrt(16)  # Utilise la fonction racine carrée de numpy
print(resultat)  # Affiche : 4.0
```

Syntaxe 2 : Importer uniquement une partie spécifique du package

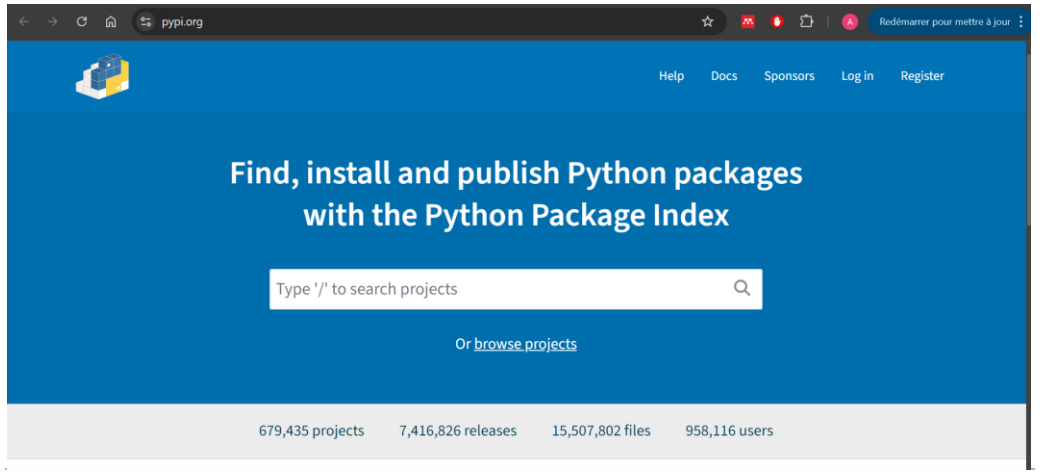
```
from numpy import sqrt
```

Cela vous permet d'utiliser directement la fonction sans utiliser l'alias :

```
from numpy import sqrt
resultat = sqrt(16)  # Pas besoin de préciser numpy
print(resultat)  # Affiche : 4.0
```

Gestion des paquets

- **pip**, le gestionnaire de package de python
- Pip utilise PyPI (Python Package Index ou l'index de paquets Python) comme source par défaut pour ses paquets. pip se trouve dans le sous dossier *Scripts* du dossier d'installation de Python.
- Pour **installer**, **désinstaller** et **mettre à jour** des paquets Python (et bien plus encore), nous utilisons **pip**
- **pip** ira chercher les dépendances du packages mais ne gérera pas les conflits de dépendances entre packages (par exemple des versions non adaptées aux différents packages installés).
- **pip** n'a pas de gestionnaire d'environnement.
- Documentation du projet PIP : https://pip.pypa.io/en/stable/



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Gestion des paquets

- Pour connaitre la version de pip installée :

  `pip --version`

- Pour connaitre la liste de toutes les paquets déjà installés, faire :

  `pip freeze` ou `pip list`

  pip list répertorie tous les paquets que vous avez installés (et leurs dépendances) dans un format intelligible

  pip freeze répertorie les paquets que vous avez installés (mais pas leurs dépendances)

- Pour installer un paquet , il suffit donc de taper dans la *fenêtre de commande* :

  `pip install <nom_du_paquet>`

- Pour obtenir des informations sur un ou plusieurs paquets

  `pip show <nom_du_paquet>`

Gestion des paquets

- **Versioning des paquets**

Le moyen le plus simple pour installer une version de paquet Python consiste à utiliser l'opérateur `==` . Par exemple, `pip install requests==2.1.0` installera, comme vous vous en doutez, la version 2.1.0 . Il existe toutefois plusieurs façons différentes de préciser la version du paquet. Par exemple :

- `pip install requests~=2.2` installera la version la plus élevée disponible au-dessus de `2.2!` , mais pas `3.0` ni les versions ultérieures.
- `pip install requests~=2.1.0` installera la version la plus élevée disponible au-dessus de `2.1.0` , mais pas la version `2.2.0` ni les versions ultérieures.
- `pip install requests>2.5.0` installera la version la plus élevée disponible au-dessus de `2.5.0` .
- `pip install "requests>2.4.0,<2.6.0"` installera la version la plus élevée disponible supérieure à `2.4.0` , mais inférieure à `2.6.0` .

Gestion des paquets

Exercice

- Ecrivez ce script et essayez de l'exécuter

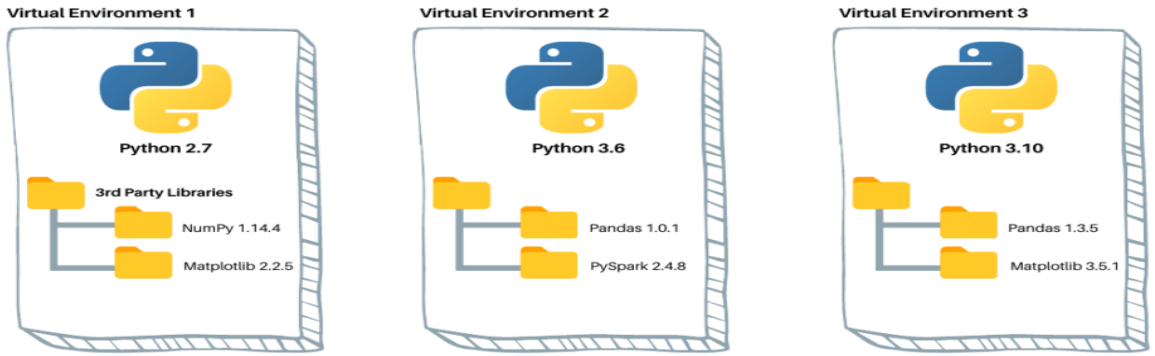
```
import boto3
import numpy
import requests
from flask import Flask


print("Congratulations. You have installed the right packages and your script has executed successfully!")
```

- S'il s'exécute correctement dès la première fois, c'est parfait, vous n'avez pas besoin d'essayer autre chose ! Cependant, si ce n'est pas le cas, vous allez devoir rechercher dans PyPI les paquets que vous devez installer, puis exécuter le script correctement !

Gestion d'environnement virtuel

- Comment faire si nous avons plusieurs projets avec des paquets différents et des versions différentes ?
  - On crée un environnement de travail local séparé de l'environnement global : un environnement virtuel
- **Un environnement virtuel** : c'est un outil utilisé pour créer des espaces isolés pour les projets Python, où vous pouvez gérer séparément les dépendances et les versions des packages pour chaque projet.
- Un environnement virtuel ou virtualenv permet de **lancer de multiples instances parallèles de l'interpréteur Python**. Chacune de ces instances à son propre ensemble de packages et ses configurations. Chaque environnement virtuel contient aussi une copie de l'interpréteur Python.
- Les packages installés dans **chaque environnement ne sont vus que dans cet environnement virtuel** et nulle part ailleurs. Même les packages les plus larges et complexes peuvent être dissociés au sein d'environnements virtuels.



Gestion d'environnement virtuel

- venv, le gestionnaire d'environnement de python
- venv est un gestionnaire d'environnement virtuel qui est installé nativement dans python. Il permet de gérer des environnements **simples**, il ne comprend pas de gestionnaire de package.
- Documentation de venv : https://docs.python.org/fr/3/library/venv.html

Environnements virtuels

- En Python, la bibliothèque `venv` permet de **créer un environnement** virtuel

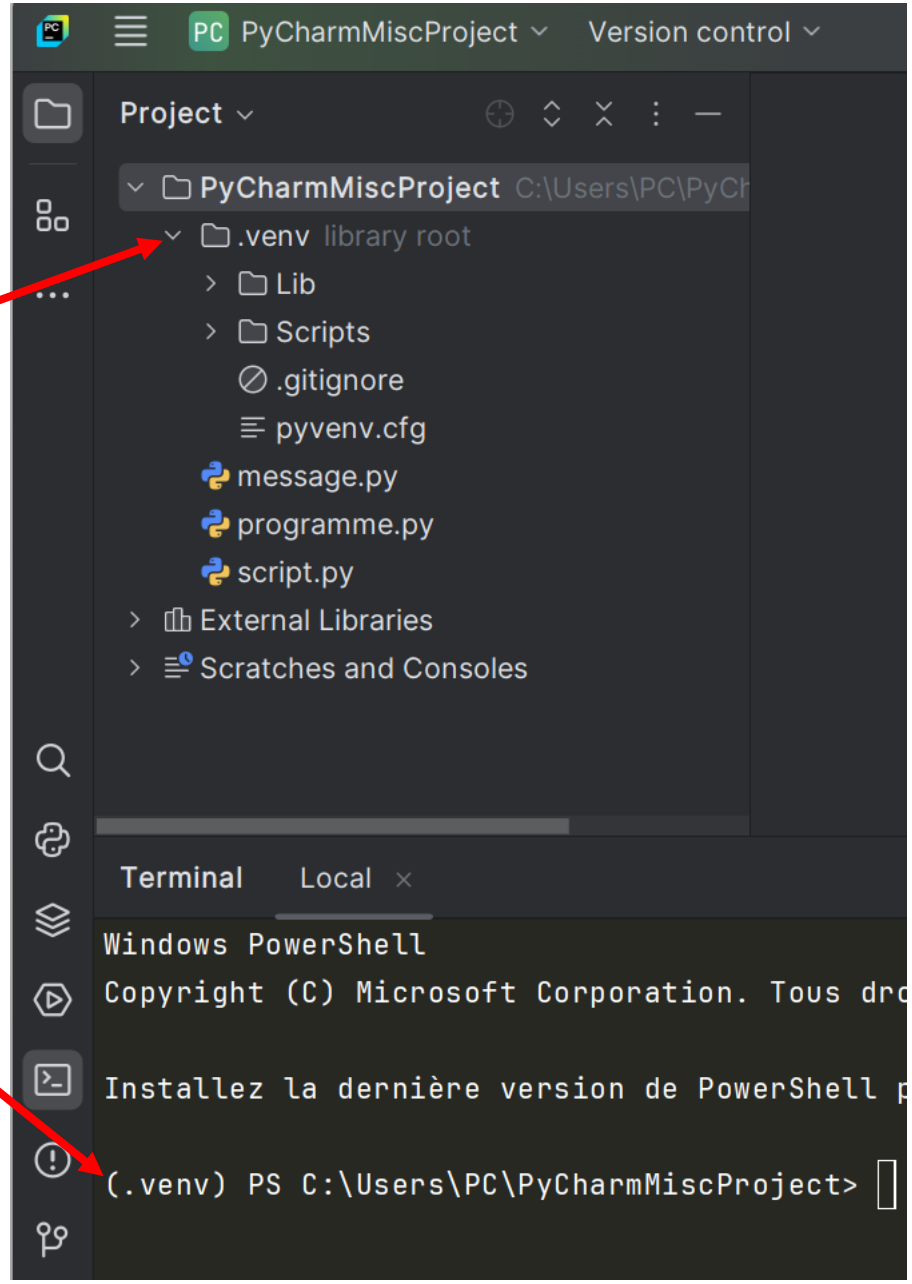
```
py -m venv /path/to/new/virtual/environment
# exemple
py -m venv monenv
```

- Python crée un **dossier monenv/** contenant :
  - une copie de l'interpréteur Python,
  - un gestionnaire pip,
  - un sous-dossier Scripts (sous Windows)
- C'est ce dossier qui permet de travailler **isolément** du Python principal de ta machine.
- Pour activer l'environnement, si vous êtes sous Windows, la commande sera `monenv\Scripts\activate.bat`
- Vous pouvez « quitter » ou « désactiver » votre environnement virtuel à l'aide de la commande `deactivate` pendant que votre environnement virtuel est activé.
- Pour supprimer un environnement virtuel, il suffit de supprimer son répertoire :

Environnement virtuel dans Pycharm

Le dossier de notre environnement virtuel

Notre terminal est par défaut avec le bon venv



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Exercice

- Soit le script suivant

```
from geopy.distance import geodesic


potential_client_one = (51.5074, 0.1278)
potential_client_two = (40.7128, 74.0060)
print('The two clients are {} miles apart'.format(
        geodesic(potential_client_one, potential_client_two).miles
```

1) Créer un environnement virtuel.
2) La première fois que vous tenterez d'exécuter l'application, elle échouera.
3) Trouver et installer les paquets dont vous pensez avoir besoin.
4) Exécuter l'application avec succès

Outils

- **conda**, un outil multilangage pour la gestion de package et d'environnement
- conda combine un gestionnaire de package et un gestionnaire d'environnements.
- Les deux composantes de conda sont :
  - **un gestionnaire de package** qui peut aller chercher des packages sur différents dépôts, notamment conda-forge, anaconda... et gérer les dépendances complexes entre packages.
  - **un gestionnaire d'environnements** permettant de créer des environnements avec des versions de python différentes et des packages dans d'autres langages que python.
- Attention, il ne faut pas confondre **conda** et **Anaconda**. Conda est un outil open source de gestion de packages et d'environnements et Anaconda est une distribution de python incluant de nombreux packages ainsi qu'un dépôt. La distribution **Anaconda** se base sur conda mais elle fixe un certain nombre de contraintes, notamment l'obligation de posséder une licence pour une utilisation professionnelle d'Anaconda et de son dépôt.

- Documentation du projet conda : https://docs.conda.io/en/latest/

Outils

- **Anaconda** est une distribution scientifique de Python : c'est-à-dire qu'en installant Anaconda, vous installerez **Python**, **Jupyter Notebook** et des dizaines de **packages scientifiques**.
- **Anaconda** est **libre et open source**. Il est destiné à la programmation dans un environnement Python et R.
- Cette distribution de Python est orientée « Data Science » et « Machine Learning »
- Cette distribution comprend entre autres :
  - Une installation de l'environnement python,
  - Des IDE (environnement de développement intégré) de dernière génération à l'instar de jupyter ou de spyder,
  - Des packages de « Data science » et « Machine learning » comme panda, numpy, scikit-learn…
  - L'outil conda pour la gestion des environnements et des répertoires de package.
- Pour commencer, téléchargez la distribution Anaconda correspondant à votre système d'exploitation, en Python version 3 : https://www.anaconda.com/download

ANACONDA

Poetry

- **Poetry**, un gestionnaire d'environnements complet pour python
- **Poetry** est un outil de gestion de dépendances et de packages pour Python. Il est plébiscité par les développeurs Python pour son approche globale du packaging de projets.

- Voici ses principales caractéristiques et fonctionnalités :
  - **Bonne Gestion des Dépendances** : Poetry, au lieu de gérer manuellement un requirements.txt, utilise un fichier spécifique pour les dépendances pyproject.toml. Celui-ci est plus structuré et lisible.
  - **Environnements Virtuels Intégrés** : Contrairement à d'autres outils, Poetry gère automatiquement les environnements virtuels pour les projets. Il crée et utilise des environnements virtuels spécifiques au projet, isolant ainsi les dépendances de projet des autres projets et de l'environnement global Python.
  - **Résolution de Dépendances Intelligente** : Poetry utilise un algorithme de résolution de dépendances sophistiqué pour s'assurer que vous avez un ensemble cohérent de dépendances. Il résout les dépendances de manière à éviter les conflits et à garantir la compatibilité.

- En résumé, Poetry est un outil complet pour la gestion de projets Python, offrant une approche unifiée pour la gestion des dépendances, le packaging, et l'isolation de l'environnement.

- Documentation Poetry : https://python-poetry.org/